PES UNIVERSITY
100 feet Ring Road, BSK 3rd Stage, Bengaluru 560085

Department of Computer Science and Engineering

(AI & ML)

B. Tech. CSE - 5th Semester

Aug – Dec 2026

UE23CS351A

DATABASE MANAGEMENT SYSTEMS (DBMS)

PROJECT REPORT on

NGO Volunteer Matchmaking System (VolunTrack.AI)

Submitted by:

Rishi J Mule
PES1UG24AM224

Rohanth Ganesha C
PES1UG24AM227

Class of Prof. Dr. Nagasundari S

[Lab Faculty]

Abstract

VolunTrack.AI is an NGO volunteer matchmaking and coordination platform that connects volunteers with community service opportunities based on skills, interests, experience, availability, and location. It addresses three problems: exact-keyword matching can miss relevant volunteers, shift registration can lead to capacity conflicts, and volunteer hours or certificates need reliable verification. The implementation combines a relational database with a vector-search component. The relational layer stores users, volunteers, NGOs, opportunities, shifts, applications, assignments, certificates, impact metrics, and audit logs, and supports constraints, indexes, views, triggers, and transactional operations. The vector component uses 384-dimensional normalized vectors and cosine similarity to support semantic matching, followed by a weighted score that also considers location, experience, and NGO verification. The web application is built with Next.js 14 and provides separate Admin, NGO, and Volunteer workflows. The repository includes SQL scripts, application/API code, database initialization, vector utilities, and automated test documentation. The supplied testing report records 21 passing checks using the SQLite WAL fallback; MySQL 8.0 is configured as the preferred engine.

Table of Contents

Sl. No	Topic	Page No.
1.	Introduction a) Problem Statement	4
2.	User Requirements Specification- Tech Stack	5
3.	E-R Model a) List of Entities [labeling Weak ones] b) List non-simple Attributes c) ER Diagram Snapshot d) ERD from tool like draw.io	6
4.	Relational Model a) List of Relations that are not Entities in ER Model b) Schema Diagram [ensure light background image]	7
5.	SQL DDL Statements	7
6.	SQL DML Statements	8
7.	NoSQL/ Vector Database Commands	9
8.	Results a) Resulting Tables' Screenshots	11
9.	Next.js-Project Tree Structure	11
10.	Few sample Screenshots of application code and demo	11
11.	Evaluation Rubrics- Completion Status	12

Introduction

1. Problem Statement

NGOs publish opportunities in different domains such as education, healthcare, environmental conservation, animal welfare, and disaster relief. A basic keyword search may fail to identify a suitable volunteer when the volunteer and NGO describe similar skills using different words. The system therefore compares volunteer profiles and opportunity descriptions using vector similarity, while retaining relational filters for operational data. Volunteer registration for shifts also requires capacity checks and duplicate-enrollment prevention. After a shift, NGO representatives verify attendance and hours; those verified records update volunteer totals and levels through database triggers. The platform also issues certificates with unique SHA-256 verification codes. The project is implemented as a web application with a relational data store and a vector search component, allowing matching and operational workflows to use the appropriate data model.

User Requirements Specification

Technology stack and justification

The project uses Next.js 14 App Router with React 18 for the web interface and API routes, JavaScript modules for backend logic, Tailwind CSS for styling, MySQL 8.0 as the configured primary relational engine, and SQLite (better-sqlite3) in WAL mode as a fallback. The database access layer selects the configured MySQL engine when reachable and otherwise falls back to SQLite. Parameterized queries are used by the data-access layer. BcryptJS is included for password hashing, and role-based access checks distinguish ADMIN, NGO, and VOLUNTEER users. The vector subsystem uses ChromaDB, with a persistent local SQLite-backed record store as a fallback when the Chroma HTTP service is unavailable.

ChromaDB is used because volunteer skills, interests, biographies, and opportunity descriptions are semi-structured text. They can be represented as dense vectors and compared by cosine similarity, which can identify related concepts even when exact words differ. The implementation defines the collection name `ngo_opportunities`, stores vector IDs with documents and metadata, and supports upsert and top-k similarity retrieval. The code also contains a persistent fallback table for embeddings, documents, and metadata.

Core user requirements: (1) volunteers can maintain their profiles and discover ranked opportunities; (2) NGOs can publish opportunities, schedule shifts, view rosters, verify attendance, and issue certificates; (3) administrators can review NGO verification and inspect platform activity; (4) shift enrollment must enforce capacity and prevent duplicates; (5) verified hours must update volunteer totals and tiers; and (6) the application must restrict role-specific operations.

Entities (no weak entities identified in the documented ER model)

The ER model is documented in PROJECT_DOCUMENTATION.md, Section 5. The relationship summary and entity list are provided here; a rendered ERD snapshot can be added if required by the evaluator.

Non-simple attributes include: volunteer skills/interests/bio; NGO address and contact details; opportunity description, required_skills and tags; volunteer profile_vector and opportunity_vector; certificate verification metadata; and audit-log change_details. The repository stores several of these as text fields rather than decomposed relational attributes.

E-R Model

- **List of Entities {highlight Weak ones}**
- **Relations corresponding to associative/relationship data rather than standalone descriptive entities**

Figure: Relational schema of VolunTrack.AI showing the ten tables, primary keys (PK), foreign keys (FK), unique constraints, and principal one-to-many relationships.

Relations and key summary (see sql/01_ddl_schema.sql for full columns and constraints):
USERS(user_id PK, email UNIQUE, role, full_name, password_hash, ...)
VOLUNTEERS(volunteer_id PK/FK→USERS, skills, interests, bio, experience_years, location_city, availability_type, total_hours_contributed, volunteer_level, profile_vector, ...)

NGOS(ngo_id PK/FK→USERS, registration_number UNIQUE, darpan_id UNIQUE, cause_category, verification_status, verified_by FK→USERS, ...)
OPPORTUNITIES(opportunity_id PK, ngo_id FK→NGOS, title, description, required_skills, location_city, status, opportunity_vector, ...)
SHIFTS(shift_id PK, opportunity_id FK→OPPORTUNITIES, shift_date, start_time, end_time, capacity, filled_count, ...)
VOLUNTEER_APPLICATIONS(application_id PK, volunteer_id FK→VOLUNTEERS, opportunity_id FK→OPPORTUNITIES, status, semantic_match_score, UNIQUE(volunteer_id, opportunity_id))
SHIFT_ASSIGNMENTS(assignment_id PK, shift_id FK→SHIFTS, volunteer_id FK→VOLUNTEERS, application_id FK→VOLUNTEER_APPLICATIONS, UNIQUE(shift_id, volunteer_id), attendance_status, hours_logged, verified_by_ngo, ...)
CERTIFICATES(certificate_id PK, certificate_code UNIQUE, volunteer_id FK, ngo_id FK, opportunity_id FK, total_hours_credited, ...)
IMPACT_METRICS(metric_id PK, opportunity_id FK→OPPORTUNITIES, ngo_id FK→NGOS, beneficiaries_served, meals_distributed, trees_planted, students_tutored, waste_cleaned_kg, ...)
AUDIT_LOGS(log_id PK, table_name, action, record_id, changed_by FK→USERS, change_details, logged_at).

Relational Model

- **List of Relations** {label those that are not Entities in ER Model}

- **Schema Diagram**

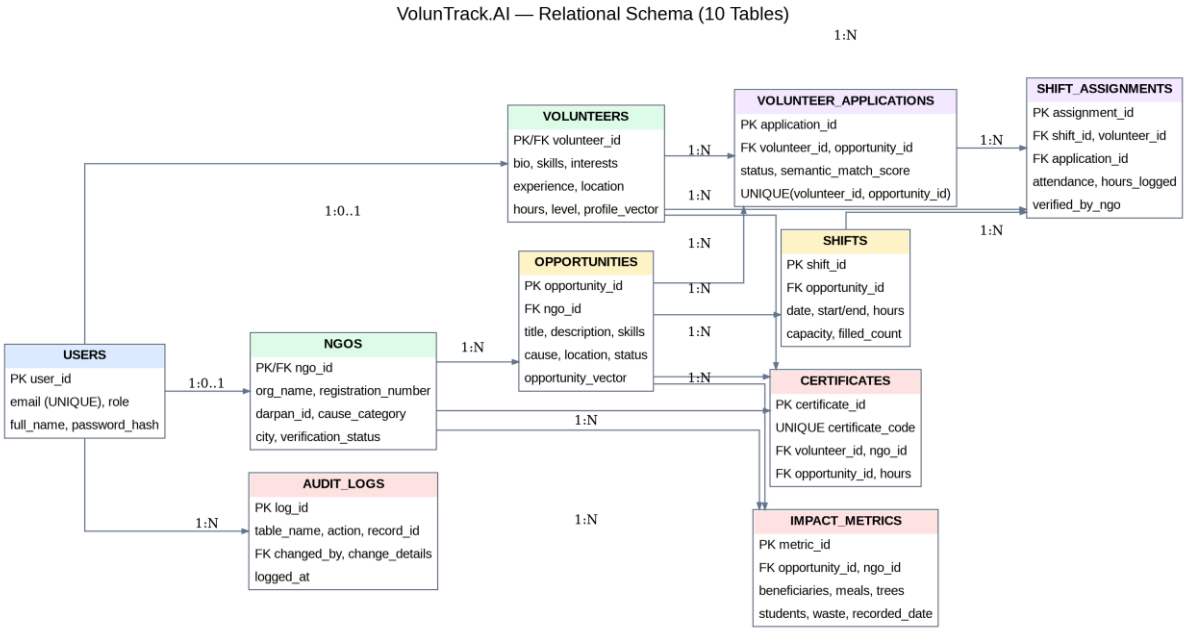


Figure: VolunTrack.AI relational schema (10 tables).

SQL DDL Statements — representative implementation

The repository's sql/01_ddl_schema.sql creates ten relational tables. The excerpts below show primary keys, a UNIQUE constraint, CHECK constraints, a foreign key with cascading delete, a default value, and an ALTER TABLE operation. The complete DDL remains in the repository.

```
-- sql/01_ddl_schema.sql
CREATE TABLE users (
    user_id VARCHAR(36) PRIMARY KEY,
    email VARCHAR(255) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role VARCHAR(20) NOT NULL
        CHECK (role IN ('ADMIN','NGO','VOLUNTEER')),
    full_name VARCHAR(150) NOT NULL,
    phone VARCHAR(20),
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE volunteers (
    volunteer_id VARCHAR(36) PRIMARY KEY,
    skills TEXT NOT NULL,
    interests TEXT NOT NULL,
    experience_years INT DEFAULT 0
        CHECK (experience_years >= 0 AND experience_years <= 60),
    location_city VARCHAR(100) NOT NULL,
    availability_type VARCHAR(30) DEFAULT 'FLEXIBLE',
    profile_vector TEXT,
    FOREIGN KEY (volunteer_id)
        REFERENCES users(user_id) ON DELETE CASCADE
);

ALTER TABLE volunteers
ADD COLUMN volunteer_level VARCHAR(30) DEFAULT 'BRONZE'
CHECK (volunteer_level IN ('BRONZE','SILVER','GOLD','PLATINUM'));
```

SQL DML Statements — representative implementation

The initialization code inserts sample users and related project records. The analytical SQL file contains SELECT operations using JOIN, LEFT JOIN, GROUP BY, aggregate functions, ORDER BY, and a correlated subquery. The repository's recorded examples are shown below; the password hash is represented by a label rather than reproducing the seeded hash.

```
-- Representative seed-data INSERT (lib/db/init.js)
INSERT INTO users
(user_id, email, password_hash, role, full_name, phone, is_active)
VALUES
('u_vol_1', 'rishi@pes.edu', '<bcrypt-hash>',
 'VOLUNTEER', 'Rishi (PES1UG24AM224)', '+91-9845012345', 1);

-- sql/06_advanced_queries.sql: JOIN + aggregation
SELECT n.org_name,
       COUNT(DISTINCT o.opportunity_id) AS total_opportunities,
       COUNT(DISTINCT sa.volunteer_id) AS active_volunteers_count,
       COALESCE(SUM(sa.hours_logged), 0.00) AS total_hours_served
FROM ngos n
INNER JOIN opportunities o ON n.ngo_id = o.ngo_id
LEFT JOIN shifts s ON o.opportunity_id = s.opportunity_id
LEFT JOIN shift_assignments sa
  ON s.shift_id = sa.shift_id
 AND sa.attendance_status = 'ATTENDED'
WHERE n.verification_status = 'VERIFIED'
GROUP BY n.ngo_id, n.org_name
ORDER BY total_hours_served DESC;

-- Correlated subquery: volunteers above their city average
SELECT v.volunteer_id, v.location_city, v.total_hours_contributed
FROM volunteers v
WHERE v.total_hours_contributed > (
    SELECT AVG(v2.total_hours_contributed)
    FROM volunteers v2
    WHERE v2.location_city = v.location_city
);
```

NoSQL / Vector Database Commands — ChromaDB

The project connects to a ChromaDB HTTP service and uses the ngo_opportunities collection. Opportunity records are stored with an ID, 384-dimensional embedding, metadata, and source document. Retrieval requests the nearest vectors using queryEmbeddings and nResults. If the HTTP service is unavailable, the code persists vectors in a local SQLite-backed fallback table named

chroma_vector_records. The 384-dimensional vectors are generated by the repository's deterministic domain-keyword, token-hash, character-bigram, and L2-normalization routine in lib/vector/embeddings.js; they are not produced by a pretrained embedding model.

```
// lib/vector/chroma.js – create/access collection and upsert
const collection = await chromaClient.getOrCreateCollection({
    name: COLLECTION_NAME
});

await collection.upsert({
    ids: [`opp-${opportunityId}`],
    embeddings: [embedding],
    metadatas: [metadata],
    documents: [document]
});

// Similarity retrieval
const results = await collection.query({
    queryEmbeddings: [queryVector],
    nResults: limit
});

// Stored record fields in the local fallback
// id, collection_name, embedding, document, metadata, created_at
```

DDL summary: sql/01_ddl_schema.sql defines ten tables and demonstrates DROP TABLE, CREATE TABLE, ALTER TABLE, primary keys, foreign keys, UNIQUE constraints, CHECK constraints, and defaults. sql/02_indexes.sql defines ten indexes for email lookup, volunteer location/availability, opportunity filtering, shift scheduling, applications, assignments, certificate lookup, impact metrics, audit logs, and NGO verification. sql/03_views.sql defines four views: vw_opportunity_details, vw_volunteer_leaderboard, vw_ngo_impact_summary, and vw_shift_occupancy.

Representative DDL:
CREATE TABLE users (user_id VARCHAR(36) PRIMARY KEY, email VARCHAR(255) NOT NULL UNIQUE, role VARCHAR(20) NOT NULL CHECK (role IN ('ADMIN','NGO','VOLUNTEER')), ...);
CREATE TABLE volunteer_applications (..., UNIQUE(volunteer_id, opportunity_id), ...);
CREATE TABLE shift_assignments (..., UNIQUE(shift_id, volunteer_id), ...);
The complete executable definitions are in sql/01_ddl_schema.sql; ellipses above indicate omitted columns and are not literal SQL.

DML and query coverage: sql/06_advanced_queries.sql includes SELECT statements with INNER/LEFT JOIN, COUNT/SUM/AVG, GROUP BY, HAVING, correlated subqueries, IN/NOT IN, and DENSE_RANK window functions. Seed INSERT statements are in sql/07_seed_data.sql, while application routes execute parameterized SELECT/INSERT/UPDATE/DELETE operations. The repository's SQL scripts also define trigger and procedure operations.

Representative query pattern:
SELECT n.ngo_id, n.org_name, COUNT(DISTINCT o.opportunity_id) AS opportunities, SUM(sa.hours_logged) AS hours
FROM ngos n JOIN opportunities o ON n.ngo_id = o.ngo_id
JOIN shifts s ON o.opportunity_id = s.opportunity_id
LEFT JOIN shift_assignments sa ON s.shift_id = sa.shift_id
GROUP BY n.ngo_id, n.org_name;
This is an illustrative pattern; use the exact full statements from sql/06_advanced_queries.sql for execution.

Vector database data model and commands: collection `ngo_opportunities`; IDs use the `opp-` prefix plus opportunity ID; each record stores a 384-dimensional embedding, source document text, and metadata. The code provides get-or-create collection, upsert, and top-k query operations through the ChromaDB client. If the HTTP service is not reachable, it persists ID, collection_name, embedding JSON, document, metadata JSON, and created_at in `chroma_data/chroma.sqlite3` table `chroma_vector_records`. The collection uses cosine distance in the configured ChromaDB behavior; similarity is exposed as 1 − distance. `lib/vector/chroma.js`, `embeddings.js`, and `similarity.js` contain the implementation.

Embedding and ranking details: lib/vector/embeddings.js creates a 384-value vector using six domain keyword clusters, deterministic token/character-bigram projections, and L2 normalization. lib/vector/similarity.js computes cosine similarity and hybrid ranking: 60% semantic similarity, 20% location alignment, 10% experience alignment, and 10% NGO verification trust. This is a deterministic domain-specific projection implemented in the repository, not a call to an external pretrained language model.

Trigger and stored-procedure coverage: sql/04_triggers.sql defines triggers to increment/decrement shift occupancy, update verified volunteer hours and level, and audit opportunity insertions and NGO verification changes. sql/05_procedures.sql defines `sp_apply_for_shift` and `sp_verify_and_issue_certificate` with transaction/rollback logic. The database access layer also exposes a transaction wrapper; stored procedure invocation is supported by its MySQL branch.

Results and verification: TESTING_REPORT.md records 21 automated checks passed and none failed (reported execution date: 1 November 2026). The report says the tested relational engine connected using SQLite WAL mode fallback. It records checks for the ten relational tables, four views, trigger registration and execution, transaction behavior, 384-dimensional normalized embeddings, ChromaDB upsert and nearest-neighbour retrieval, bcrypt/RBAC checks, and analytical SQL. These are repository-reported test results; this document has not independently rerun the test suite.

Selected seed data in lib/db/init.js includes volunteer profiles, NGO profiles, opportunities, shifts, applications, assignments, certificates, impact metrics, and audit entries. The supplied TESTING_REPORT.md lists sample query outcomes. Add terminal and database screenshots from a fresh run to substantiate the results in the final submission.

Next.js project tree (selected):
app/ (pages and route handlers): page.js, login/page.js, opportunities/page.js, volunteer/dashboard/page.js, ngo/dashboard/page.js, admin/dashboard/page.js, verify/[code]/page.js; app/api/ (auth, opportunities, shifts, certificates, NGOs, volunteers, admin); components/Navbar.js; lib/auth.js; lib/db/{index.js,init.js}; lib/vector/{chroma.js,embeddings.js,similarity.js};

sql/{01_ddl_schema.sql,02_indexes.sql,03_views.sql,04_triggers.sql,05_procedures.sql,06_advanced_queries.sql, 07_seed_data.sql}; scripts/{seed.js,test-db.js,verify-runtime.js}. package.json identifies Next.js 14, React 18, mysql2, better-sqlite3, chromadb, and bcryptjs.

Application/demo evidence: the repository's DEMONSTRATION_GUIDE.md describes the homepage, opportunity explorer and AI match action, volunteer dashboard, NGO dashboard and attendance roster, certificate issue/verify route, and admin dashboard with SQL Lab. The current template does not contain application screenshots; capture genuine screenshots during the live demo and insert them here. Code examples may be taken from app/api/opportunities/match/route.js, lib/vector/chroma.js, and sql/04_triggers.sql.

Completion status (based on repository evidence): Database design — implemented and documented; SQL database creation — schema, indexes, views, triggers and procedures provided; SQL operations — advanced query script and application DML present; advanced concepts — triggers, procedures, views and vector search present; web application integration — Next.js pages and API handlers present; authentication/authorization — three roles and bcrypt/RBAC code present; documentation/testing/demo — multiple markdown reports and a 21-check test report present. The rubric calls out CTE/recursive SQL, two roles minimum, screenshots, and individual contributions; the reviewed sources do not explicitly document a CTE/recursive query or provide embedded application screenshots. These should be verified/added before marking the entire submission complete.

Evaluation Rubrics

#	Criteria	Distribution	Status
1	Database Design	ER diagram , relational schema with keys, relationships, and constraints	Implemented; ER model and 10 relations documented
2	Database Creation	SQL database implementation - DDL with constraints , NoSQL/Vector database creation , Technology justification	Implemented; MySQL configured, SQLite WAL fallback, ChromaDB adapter
3	SQL Operations and Queries	DML operations , Nested/correlated/Recursive SQL queries, Join SQL queries	Implemented; joins, aggregation, HAVING, correlated subquery, window functions
4	Advanced Database Concepts	Advanced database concepts (Procedure/function, trigger, CTE) , NoSQL/Vector feature RUD	Implemented; views, triggers, stored procedures, vector upsert/query; CTE not evidenced
5	Web Application and Database Integration	Next.js interface , Backend/API and database integration	Implemented; Next.js pages and API integration present

#	Criteria	Distribution	Status
6	Authentication and Authorization	Authentication implementation (front end and backend) and minimum two roles and role-based access control in database	Implemented; ADMIN, NGO, VOLUNTEER roles and RBAC checks documented
7	Documentation, Git/GitHub, Demonstration, Testing, and Originality	Documentation, Git/GitHub , Originality and individual contribution	Partially evidenced; docs/tests/demo guide present; screenshots and contribution proof should be added